

6. Write a Java program to implement the producer-consumer problem using threads.

Type	Input	Expected Output	Explanation
Success	Produce item 1, consume item 1	Produced and consumed correctly	Basic synchronization
Success	Produce items 1 to 5	All items consumed in order	Multiple operations
Success	Buffer size = 1	Producer/consumer waits correctly	wait-notify validation
Failure/Edge	Consumer starts before producer	Consumer waits, no crash	Empty buffer case
Failure/Edge	No synchronization	Race condition may occur	Breaks incorrect logic

Algorithm:

1. Start.
2. Create a shared Buffer object.
3. Initialize `available` to `false`.
4. Create Producer and Consumer threads using the same Buffer.
5. Producer checks whether the buffer is full.
6. If full, Producer calls `wait()`.
7. Otherwise, Producer stores an item and sets `available = true`.
8. Producer calls `notify()`.
9. Consumer checks whether the buffer is empty.
10. If empty, Consumer calls `wait()`.
11. Otherwise, Consumer consumes the item and sets `available = false`.
12. Consumer calls `notify()`.
13. Repeat steps 5–12 until 5 items are produced and consumed.
14. Stop.

Program:

```
class Buffer {
    private int item;
    private boolean available = false;
    // Produce an item
    public synchronized void produce(int value) {
        while (available) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        item = value;
        available = true;
        System.out.println("Produced: " + item);
        notify();
    }
    // Consume an item
    public synchronized int consume() {
```

```

while (!available) {
    try {
        System.out.println("Consumer waits, no crash");
        wait();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
available = false;
System.out.println("Consumed: " + item);
notify();
return item;
}
}

class Producer extends Thread {
    private Buffer buffer;

    public Producer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <=5; i++) {
            buffer.produce(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

class Consumer extends Thread {
    private Buffer buffer;

    public Consumer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <=5; i++) {
            buffer.consume();
            try {
                Thread.sleep(800);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```
    }  
}  
public class ProducerConsumer {  
    public static void main(String[] args) {  
        Buffer buffer = new Buffer();  
  
        Producer producer = new Producer(buffer);  
        Consumer consumer = new Consumer(buffer);  
  
        //producer.start();  
        consumer.start();  
        producer.start();  
    }  
}
```